Rival Payments — Integration Guide

This is the single reference for integrating with the Rival Payments platform. There are **two independent integration directions**, and you typically use one or the other depending on who you are:

Part	Direction	Who it's for	What it does
A — Ingest API	**You → Rival**	Transaction **sources** (Whish, OMT, banks, …)	Push transactions *into* the platform.
B — CRM Webhooks	**Rival → You**	**Companies** on the platform	Receive each transaction's status in *your own* CRM.

- **A source** (e.g. Whish) reads **Part A** to learn how to POST transactions to us.
- **A company** reads **Part B** to learn how to receive those transactions, live, in their CRM.

Connection basics (both parts)

Item	Value
Base URL	`https://portal.rivalpayments.com/v1`
Protocol	HTTPS only
Content type	`application/json` (request and response)
Encoding	UTF-8
Money amounts	Always **decimal strings** (e.g. `"150.00"`), never floats
Currency	`USD` (only supported currency today)

Response envelope. Every JSON response from an API endpoint you **call** (the Ingest API in Part A, and the company endpoints in Part B like `/company/crm/config`) uses one shape, so you read the status code from the body and branch on `success`:

```
// success — your payload is always under "data"
{ "success": true, "statusCode": 200, "data": { /* … */ } }

// error — the failure is under "error"
{ "success": false, "statusCode": 400, "error": { "code": "VALIDATION_ERROR", "message":
"…", "details": {} } }
```

`statusCode` mirrors the HTTP status. The Part A examples below show the full envelope. (The **Part B** CRM webhook body we POST to *your* endpoint is a separate payload and is **not** wrapped

in this envelope.) A table of the error codes you may see is in the Appendix.

Part A — Ingest API (for transaction sources)

Use this if you are a **source** that needs to push transactions into Rival Payments (this is what Whish, OMT, etc. use). The flow is:

1. **Discover** the operating entities you can post for → `GET /ingest/companies`.
2. **Submit** a transaction for one of them → `POST /ingest/{sourceCode}/transactions`.

A.0 Authentication

Every request carries a **source API key** (issued to you by Rival Payments) as a Bearer token:

```
Authorization: Bearer tsk_<prefix>_<secret>
```

- The key identifies your **source**. The URL path also names your source (`{sourceCode}`); the key must be bound to that same source or the request is **403**.
- Keep the key secret — send it only over HTTPS, never embed it client-side.
- **Rate limit: 300 requests / rolling 60 s** per key. Over the limit → **429** `RATE_LIMITED`; back off and retry.

A.1 Discover operating entities — `GET /ingest/companies`

Returns every operating entity that is currently **active and visible** to sources. Cache the `operatingId`s and refresh periodically.

Request

```
GET /v1/ingest/companies
Authorization: Bearer <SOURCE_KEY>
```

Response — `200 OK`

```
{
  "success": true,
  "statusCode": 200,
  "data": {
    "data": [
      {
        "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
        "slug": "acme-retail",
        "name": "ACME Retail",
        "defaultCurrency": "USD"
      },
      {
        "operatingId": "b14d77e2-1f53-49c0-9e8a-2d6c0f3b9a44",
        "slug": "globex-fuel",
        "name": "Globex Fuel",
        "defaultCurrency": "USD"
      }
    ]
  }
}
```


The list payload `{ "data": [...] }` sits under the envelope's `data`, so the entities are at **`data.data`**.

Field	Type	Description
`data.data[].operatingId`	string (UUID)	**The value you send as `operatingId` when submitting a transaction.** Stable — safe to cache.
`data.data[].slug` / `name`	string	Human-friendly identifiers, for display only.
`data.data[].defaultCurrency`	string	The entity's default settlement currency (`USD`).

A.2 Submit a transaction — `POST /ingest/{sourceCode}/transactions`

`{sourceCode}` is your fixed source segment (e.g. `whish`). It must match your API key, or the request is **403**.

Request body

```
{
  "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
  "externalTransactionId": "WISH-2026-000123456",
  "amount": "150.00",
  "currency": "USD",
  "payload": {
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish"
  }
}
```

Field	Type	Required	Rules
`operatingId`	string (UUID)	**Yes**	An `operatingId` from `GET /ingest/companies`. Must be active and visible.
`externalTransactionId`	string	**Yes**	Your unique reference (1–200 chars). Used for de-duplication.
`amount`	string (decimal)	**Yes**	Positive decimal string, ≤8 decimals / ≤12 integer digits.
`currency`	string	**Yes**	`USD`.
`payload`	object	No (`{}`)	Descriptive fields (below). Extra keys are stored verbatim.
`idempotencyKey`	string	No	Rarely needed — `externalTransactionId` already de-duplicates. 8–120 chars.

`payload` recognised keys (all optional): `customer` (payer), `tradingAccount` (recipient account), `phone`, `note`. Any other keys are preserved for audit.

Response — `201 Created` (transaction object under `data`)

```
{
  "success": true,
  "statusCode": 201,
  "data": {
    "id": "a7c3e9f1-2b44-4d8e-9c10-5f6a7b8c9d0e",
    "operatingId": "3f8a1c20-9b2e-4d77-8a1f-7c2e5b0d4a11",
    "sourceId": "5d2b1a90-7e34-4c11-b8f2-1a9c3e7d6b50",
    "sourceCode": "whish",
    "externalTransactionId": "WISH-2026-000123456",
    "amount": "150.00",
    "currency": "USD",
    "status": "COMPLETED",
    "customer": "John Doe",
    "tradingAccount": "0096170123456",
    "phone": "0096170123456",
    "note": "Top-up via Whish",
    "processedAt": "2026-06-04T12:34:56.000Z",
    "createdAt": "2026-06-04T12:34:56.000Z"
  }
}
```


> The source-facing response deliberately omits any platform fee/commission figures.

Idempotency: the pair **(your source, `externalTransactionId`)** de-duplicates automatically — resending the same request returns the existing transaction and books no new money, so retries are safe. Reusing the same `externalTransactionId` for a *different* entity is rejected (**409**).

Part B — CRM Webhooks (for companies)

Use this if you are a **company** on the platform and want every transaction's status to appear **live in your own CRM**. Rival Payments makes an outbound HTTPS `POST` to an endpoint you provide, for each step of a transaction's lifecycle.



B.1 Connect your CRM

You give Rival two things — your CRM's **endpoint URL** and an **API key** your CRM expects — and we start delivering events. Two ways to set it:

Option 1 — Dashboard (recommended). Sign in to the company portal → **CRM** page → enter your endpoint URL + API key → save. (Company **owner** role required.)

Option 2 — API.

```
PUT /v1/company/crm/config
Authorization: <company session cookie>      # JWT cookie auth
x-csrf-token: <token>
Content-Type: application/json

{
  "apiUrl": "https://your-crm.example.com/rival/webhook",
  "apiKey": "whatever-secret-your-crm-expects"
}
```

Saving **enables** delivery immediately. To check or stop:

- `GET /v1/company/crm/config` → `data: { apiUrl, hasApiKey, enabled, demoUrl }` (the key itself is never returned).
- `DELETE /v1/company/crm/config` → disables delivery and clears the URL/key.

Endpoint requirements

Your `apiUrl` is validated when you save it (and again before every send):

- **Must be `https://`.**
- **Must be publicly reachable** — private, loopback, link-local, and cloud-metadata addresses (e.g. `10.0.0.0/8`, `127.0.0.1`, `169.254.169.254`) are rejected.
- Must not embed credentials in the URL (`https://user:pass@…` is rejected).

An endpoint that later resolves to an internal address is failed at send time (DNS-rebind protection).

B.2 What we send

For each event we `POST` to your `apiUrl`:

Headers

Header	Value
`Content-Type`	`application/json`
`Authorization`	`Bearer <your apiKey>` — the key you configured. **Verify this.**
`x-crm-event`	the event type, e.g. `transaction.completed`
`x-crm-timestamp`	Unix seconds when we sent the attempt

Body


```
{
  "event": "transaction.completed",
  "reference": "whish:10231",
  "sentAt": "2026-06-04T12:00:00.000Z",
  "transaction": {
    "id": "0e1c…",
    "status": "COMPLETED",
    "source": "whish",
    "externalTransactionId": "10231",
    "amount": "25.00",
    "currency": "USD",
    "netAmount": "24.50",
    "commissionAmount": "0.50",
    "customer": "John Doe",
    "tradingAccount": null,
    "phone": "+961…",
    "note": "Order #1234",
    "occurredAt": "2026-06-04T12:00:00.000Z"
  }
}
```

Field	Type	Description
`event`	string	One of the event types.
`reference`	string	**Stable id for the logical transaction** — `"{source}:{externalTransactionId}"` (e.g. `whish:10231`). **Upsert by this.**
`sentAt`	string (ISO 8601)	When this delivery attempt was sent.
`transaction.id`	string (UUID) | null	Ledger transaction id. **`null`** while a Whish link is still `PENDING` (no ledger row yet); a UUID once `COMPLETED`.
`transaction.status`	string	`PENDING` · `COMPLETED` · `FAILED` · `EXPIRED` · `REVERSED`.
`transaction.source`	string	Source code, e.g. `whish`, `omt`.
`transaction.externalTransactionId`	string	The source's own reference.
`transaction.amount`	string (decimal)	Gross amount.
`transaction.currency`	string	`USD`.
`transaction.netAmount`	string | null	Amount credited to your wallet after fee. **`null` until `COMPLETED`** (fee not yet known).
`transaction.commissionAmount`	string | null	Platform fee. **`null` until `COMPLETED`.**
`transaction.customer`	string | null	Payer name (if provided).
`transaction.tradingAccount`	string | null	Recipient account (if provided).
`transaction.phone`	string | null	Contact phone (if provided).
`transaction.note`	string | null	Free-text note (if provided).
`transaction.occurredAt`	string (ISO 8601)	When this status occurred.

> **Note:** unlike the source-facing Ingest API, the CRM payload **does** include `netAmount` and `commissionAmount` — this is *your* company's data, so you see exactly what was credited and what fee applied.

Event types and lifecycle

`event`	`status`	**Meaning**
`transaction.pending`	`PENDING`	A payment was initiated (e.g. a Whish link created) — not settled yet.
`transaction.completed`	`COMPLETED`	Settled; your wallet was credited `netAmount`.
`transaction.failed`	`FAILED`	The payment failed.
`transaction.expired`	`EXPIRED`	A pending payment expired unpaid.
`transaction.reversed`	`REVERSED`	A completed transaction was reversed.

One record, evolving. All events for the same logical transaction share the same `reference`, so a Whish payment arrives as `pending` → `completed` (and possibly `failed` / `expired` / `reversed`). **Upsert by `reference`** and let the latest `status` win — don't create a new record per event.

B.3 Acknowledging & retries

- **Acknowledge with any `2xx`.** That marks the event delivered.
- **We retry transient failures** — a network error/timeout, `429`, or any `5xx` — with growing backoff: **1 min → 5 min → 15 min → 1 h → 6 h** (up to **6 attempts** total, including the first immediate one). After that the event is marked failed.
- **We do *not* retry `4xx`** (`400/401/403/404/409/422`). Those mean the request itself is wrong (usually auth/config), so retrying wouldn't help — fix the config instead.
- **Per-attempt timeout is 5 s.** Respond quickly: ack first, do heavy work async.
- Delivery is **durable** — events are queued, so a brief outage on your side won't drop them; a background sweep re-delivers what's due.

Because of retries, **expect occasional duplicates and out-of-order arrivals.** Make your handler **idempotent** (keyed on `reference` + `status`); use `occurredAt` / `status` rather than assuming order.

B.4 Securing your endpoint

- **Verify the `Authorization: Bearer` header** equals the API key you configured. Reject anything else with `401`.
- **Require HTTPS** (we only ever send over TLS).
- Optionally reject requests whose `x-crm-timestamp` is older than a few minutes to limit replay.

B.5 Try it first — the demo CRM

Before wiring your real CRM, you can enable a **built-in demo inbox** from the dashboard (or `POST /v1/company/crm/config` demo enable). The platform then delivers events to a first-party

preview inbox you can watch live — a zero-setup way to see the exact payloads and lifecycle for your company's transactions.

Appendix — Error codes

HTTP	`code`	When
`400`	`VALIDATION_ERROR`	Request body/params failed validation.
`401`	`UNAUTHORIZED`	Missing/invalid/revoked credential.
`403`	`FORBIDDEN`	Key not bound to the URL's source, entity not available, or wrong key type.
`409`	`CONFLICT`	Duplicate `externalTransactionId` across entities, or a write conflict (retry).
`429`	`RATE_LIMITED`	Per-key throttle exceeded (Ingest API).
`5xx`	`INTERNAL_ERROR`	Unexpected server error — retry with backoff.

Appendix — Glossary

Term	Meaning
Source	A system that pushes transactions in (Whish, OMT, …), authenticated by a source API key.
operatingId	The UUID identifying a company/operating entity in the Ingest API.
tradingAccount	The recipient account on a transaction (same field across Ingest and CRM).
reference	`"{source}:{externalTransactionId}"` — the stable id to upsert a CRM record on.
Company	A tenant on the platform that owns transactions and can receive CRM webhooks.

Rival Payments — Integration Guide. For partner-specific onboarding (e.g. your issued source key), see the onboarding sheet provided to you.